

OPERAZIONI INSIEMISTICHE - ESEMPIO

- Fare una lista dei numeri di progetti per i progetti che coinvolgono un impiegato il cui cognome è 'Smith' come lavoratore **oppure** come manager del dipartimento che controlla il progetto:

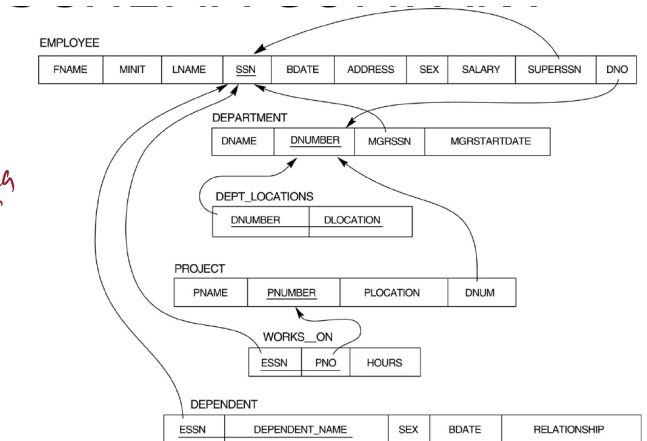
```
(SELECT PNUMBER
FROM PROJECT, WORKS_ON, EMPLOYEE
WHERE PNUMBER=PNO AND ESSN=SSN AND
      LNAME='Smith');

UNION

(SELECT PNUMBER
FROM PROJECT, DEPARTMENT, EMPLOYEE
WHERE DNUM=DNUMBER AND MGRSSN=SSN AND
      LNAME='Smith')
```

UNION
INTERSECT
EXCEPT

Schema
Azienda



QUERY ANNIDATE E CONFRONTO DI INSIEMI

- La query usata per mostrare la **UNION** può essere così riformulata:

```
SELECT DISTINCT PNUMBER
FROM PROJECT
WHERE PNUMBER IN
  (SELECT PNUMBER
   FROM PROJECT, DEPARTMENT, EMPLOYEE
   WHERE DNUM=DNUMBER AND MGRSSN=SSN AND LNAME='Smith')

OR

PNUMBER IN
  (SELECT PNO
   FROM WORKS_ON, EMPLOYEE
   WHERE ESSN=SSN AND LNAME='Smith');
```

query
annidate

QUERIES ANNIDATE E CONFRONTO DI INSIEMI (2)

- La prima query seleziona il numero dei progetti che hanno uno 'Smith' come manager, mentre la seconda seleziona il numero di progetto dei progetti che hanno uno 'Smith' come lavoratore.
- Nella query esterna selezioniamo una tupla PROJECT se il valore PNUMBER compare nel risultato di una delle query annidate.

L'OPERATORE IN

- L'operatore **IN** confronta un valore con un insieme di tuple union-compatibili.
- L'operatore **IN** permette di specificare valori multipli nella clausola WHERE.

Esempio:

```
SELECT DISTINCT ESSN
FROM WORKS_ON
WHERE (PNO, HOURS) IN
  (SELECT PNO, HOURS
   FROM WORKS_ON
   WHERE ESSN='123456789');
```

Funziona anche con
più valori

ALTRI OPERATORI

■ ANY (o SOME)

- confronta un singolo valore (attributo) **v** con un multiset **V**, restituendo TRUE se **v** è uguale a qualche valore in **V**.
- ANY** e **SOME** sono equivalenti. Possono essere combinati con **>**, **≥**, **<**, **≤**, **<>**. *operatori di confronto*
- = **ANY** è equivalente ad usare l'operatore **IN**.

■ Anche **ALL** può essere combinato con questi operatori.

- v > ALL V** è TRUE se il valore **v** è maggiore di tutti i valori in **V**.

OPERATORE ALL - ESEMPIO

- Trovare tutti i nomi degli impiegati il cui salario è maggiore del salario di tutti gli impiegati del dipartimento 5:

```
SELECT LNAME, ENAME
FROM EMPLOYEE
WHERE SALARY > ALL (SELECT SALARY
                     FROM EMPLOYEE
                     WHERE DNO=5);
```

OPERATORE ANY - ESEMPIO

```
SELECT LNAME, ENAME
FROM EMPLOYEE
WHERE SALARY > ANY (SELECT SALARY
                    FROM EMPLOYEE);
```

*migliore di almeno uno
dei valori restituiti da
questa query annidata*

*risult. query
annidata*

- Trovare tutti i nomi degli impiegati tranne quelli il cui salario è minimo.

*tutti tranne
questo*

1000
3000
7500
500
10000

CASI DI AMBIGUITÀ

■ Ambiguità nei nomi di attributi:

- Si ha, se esistono attributi con lo stesso nome, uno in una relazione nella clausola FROM della query esterna e l'altro in una relazione della clausola FROM della query interna.

- Regola:** un riferimento a un attributo non qualificato riferisce alla relazione dichiarata nella query annidata più interna.

CASI DI AMBIGUITÀ - ESEMPIO

- Trovare il nome di ogni impiegato che ha una persona a carico con lo stesso nome e lo stesso sesso dell'impiegato:

```
SELECT E.FNAME, E.LNAME
FROM EMPLOYEE AS E
WHERE E.SSN IN ( SELECT ESSN
                FROM DEPENDENT
                WHERE ESSN=E.SSN
                  AND DEPENDENT_NAME = E.FNAME
                  AND SEX=E.SEX);
```

È necessario qualificarlo altrimenti farebbe riferimento alla relazione DEPENDENT.

$R(A1, A1, A3)$

```
SELECT R1.A1
FROM R AS R1, R AS R2
WHERE ...
```

La ambiguità può
persistere anche
senza query annidate

CASI DI AMBIGUITÀ (2)

- In generale, una query scritta con blocchi annidati SELECT...FROM...WHERE e con operatori di confronto = o IN può essere sempre espressa come un singolo blocco.

- La precedente query può anche essere scritta come:

```
SELECT E.FNAME, E.LNAME
FROM EMPLOYEE AS E, DEPENDENT AS D
WHERE E.SSN=D.SSN AND E.SEX=D.SEX AND
E.FNAME=D.DEPENDENT_NAME
```

L'OPERATORE CONTAINS

- L'implementazione originale SQL su **systemR** prevedeva un operatore **CONTAINS** per confrontare due insiemi.
- È stato poi eliminato per motivi di efficienza.

L'OPERATORE CONTAINS - ESEMPIO

- Ritrovare il nome e cognome di ciascun impiegato che lavora su tutti i progetti controllati dal dipartimento 5:

```
SELECT FNAME, LNAME
FROM EMPLOYEE AS E
WHERE (( SELECT PNO
        FROM WORKS ON
        WHERE E.SSN=ESSN)
      CONTAINS
      ( SELECT PNUMBER
        FROM PROJECT
        WHERE DNUM=5));
```

$\subseteq$ contenimento

EXISTS E NOT EXISTS

▪ EXISTS e NOT EXISTS:

- Per verificare se il risultato di una query annidata correlata è vuota.

▪ **Esempio:** Ritrovare il nome degli impiegati che non hanno persone a carico:

```
SELECT FNAME, LNAME
FROM EMPLOYEE
WHERE NOT EXISTS ( SELECT *
                   FROM DEPENDENT
                   WHERE SSN=ESSN);
```

*Si può formulare con
NOT IN*

INSIEMI ESPLICITI

▪ Trovare il SSN di tutti gli impiegati che lavorano sui progetti 1, 2 o 3:

```
SELECT DISTINCT ESSN
FROM WORKS_ON
WHERE PNO IN (1, 2, 3);
```

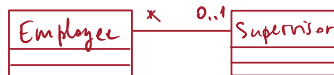
WHERE PNO=1 OR PNO=2 OR PNO=3;

▪ Si può anche testare se un valore è **NULL**:

- = e ≠ sono scritti come 'IS' e 'IS NOT' per confronti con NULL.

▪ **Esempio:** Trovare il nome di tutti gli impiegati che non hanno supervisori:

```
SELECT FNAME, LNAME
FROM EMPLOYEE
WHERE SUPERSSN IS NULL;
```



Esiste anche IS NOT NULL

LA KEYWORD AS

▪ È possibile rinominare qualsiasi attributo che compare in una query con la keyword **AS**:

```
SELECT E.LNAME AS EMPLOYEE_NAME,
       S.LNAME AS SUPERVISOR_NAME
FROM EMPLOYEE AS E, EMPLOYEE AS S
WHERE E.SUPERSSN=S.SSN;
```

SQL è un linguaggio dichiarativo. Esso specifica il risultato delle query ma non come tale risultato deve essere ottenuto processando la query da parte del DBMS. Come conseguenza le query possono essere formulate in modo abbastanza naturale.

```
-- Creazione delle tabelle
CREATE TABLE Fornitori (
  fid      INT PRIMARY KEY,
  fnome    VARCHAR(50) NOT NULL,
  indirizzo VARCHAR(100)
);

CREATE TABLE Pezzi (
  pid      INT PRIMARY KEY,
  pname    VARCHAR(50) NOT NULL,
  colore   VARCHAR(20),
  peso     DECIMAL(5,2)
);

CREATE TABLE Catalogo (
  fid      INT,
  pid      INT,
  costo    DECIMAL(8,2),
  PRIMARY KEY (fid, pid),
  FOREIGN KEY (fid) REFERENCES Fornitori(fid),
  FOREIGN KEY (pid) REFERENCES Pezzi(pid)
);
```

Fornitori(*fid*, *fnome*, *indirizzo*)
Pezzi(*pid*, *pnome*, *colore*, *peso*)
Catalogo(*fid*, *pid*, *costo*)

In classe: nomi dei fornitori che forniscono
almeno un pezzo rosso

```
-- Dati di esempio per Fornitori
INSERT INTO Fornitori (fid, fnome, indirizzo) VALUES
  (1, 'Sapienza', 'Via Cavour, 18'),
  (2, 'Tor Vergata', 'Via Columbia, 2'),
  (3, 'RomaTre', 'Viale Ostiense, 159'),
  (4, 'Polito', 'Corso Duca degli Abruzzi, 24');

-- Dati di esempio per Pezzi
INSERT INTO Pezzi (pid, pname, colore, peso) VALUES
  (10, 'Vite M4', 'rosso', 0.05),
  (11, 'Bullone M8', 'verde', 0.12),
  (12, 'Dado M6', 'rosso', 0.03),
  (13, 'Perno Ø5', 'blu', 0.20),
  (14, 'Rondella 5', 'verde', 0.02);

-- Dati di esempio per Catalogo
INSERT INTO Catalogo (fid, pid, costo) VALUES
  (1, 10, 0.10),
  (1, 11, 0.15),
  (1, 12, 0.08),
  (2, 10, 0.12),
  (2, 13, 0.25),
  (3, 11, 0.14),
  (3, 12, 0.09),
  (3, 14, 0.05),
  (4, 10, 0.11),
  (4, 14, 0.06);
```

SQLite: DBMS leggero e semplice, con GUI intuitiva (non ha procedure)

pgadmin: uno dei frontend di PostgreSQL.

$\pi(\sigma_{\dots}(\text{Fornitori} \bowtie (\text{Catalogo} \bowtie \text{Pezzi})))$

Esercizio 1

Domanda

Nomi dei fornitori che forniscono pezzi rossi

Algebra relazionale

$\pi_{fnome} \left(\text{Fornitori} \bowtie_{\text{Fornitori.fid}=\text{Catalogo.fid}} \left(\sigma_{\text{colore}='rosso'}(\text{Pezzi}) \bowtie_{\text{Pezzi.pid}=\text{Catalogo.pid}} \text{Catalogo} \right) \right)$

La selezione si può
fare anche alla
fine (all'esterno)